PCB Defect Detection and Classification System

Introduction

1.1. Project Overview

Printed Circuit Boards (PCBs) are fundamental components in virtually all electronic systems, ranging from consumer electronics and communication devices to aerospace and medical equipment. The reliability and performance of these systems heavily depend on the structural and electrical integrity of PCBs. During manufacturing, however, defects may arise due to misalignment, improper etching, drilling errors, material inconsistencies, or environmental factors. Common PCB defects include open circuits, short circuits, missing holes, copper spurs, mouse bites, and spurious copper traces. If left undetected, these defects can result in product failure, increased production costs, and compromised safety.

Traditional PCB inspection techniques involve manual visual inspection or semi-automated optical inspection systems. Manual inspection is labor-intensive, subjective, and prone to inconsistencies due to human fatigue and variability. Automated Optical Inspection (AOI) systems improve reliability but can be expensive and require extensive calibration. Moreover, many systems struggle with detecting subtle defects under varying lighting conditions.

The objective of this project is to design and implement a fully automated PCB defect detection and classification system that integrates image processing techniques with deep learning-based classification. The system follows a reference-based inspection approach, where a defect-free template image is compared with a defective test image. Image subtraction is employed to highlight differences, followed by contour detection to localize defect regions. A convolutional neural network based on EfficientNet-B0 is then used to classify each detected defect into predefined categories.

The final system is deployed using a Streamlit web application that allows users to upload PCB images and receive annotated outputs in real time. This integration ensures that the system is not only accurate but also user-friendly and suitable for practical industrial implementation.

1.2. System Architecture

The proposed system follows a structured multi-stage pipeline:

1. Template and Test Image Upload
2. Image Preprocessing and Subtraction
3. Defect Mask Generation
4. Contour Detection
5. ROI Extraction
6. Deep Learning Classification
7. Annotated Output Generation
8. Result Export

The backend is modularized into independent processing units, while the frontend is implemented using Streamlit for real-time interaction.

This architecture ensures separation of concerns, maintainability, and scalability.

1.3. Defect Categories

The system classifies PCB defects into six categories based on the DeepPCB dataset:

1. Missing Hole – Absence of drilled hole in PCB
2. Mouse Bite – Irregular copper erosion along edges
3. Open Circuit – Broken conductive trace
4. Short – Unintended connection between traces
5. Spur – Unwanted copper protrusion
6. Spurious Copper – Extra copper deposition

Each defect type presents unique structural characteristics, requiring careful preprocessing and classification strategies.

Milestone 1: Dataset Preparation and Image Processing

Module 1: Dataset Setup and Reference-Based Image Subtraction

1. Objective

The primary objective of this module is to prepare the DeepPCB dataset for processing and implement a robust reference-based defect localization method using image subtraction techniques. Accurate defect localization at this stage is critical, as it directly influences the quality of ROI extraction and classification in subsequent modules.

2. Dataset Organization and Preparation

The DeepPCB dataset consists of paired images, including defect-free template images (golden boards) and defective test images. Proper dataset organization was essential to ensure reproducibility and systematic processing.

The dataset was structured into category-wise directories corresponding to the six defect classes. This structured approach ensured:

- Efficient data loading
- Clear ground-truth labeling
- Scalable training pipeline

Before processing, image consistency checks were performed to verify resolution alignment and image integrity.

3. Image Alignment and Preprocessing

Since subtraction requires pixel-wise comparison, both template and test images were resized to identical dimensions. Images were converted to grayscale to reduce computational complexity while preserving structural trace patterns.

To enhance contrast and improve subtle defect visibility, Contrast Limited Adaptive Histogram Equalization (CLAHE) was applied. CLAHE enhances local contrast while limiting noise amplification, making it particularly effective for detecting faint copper irregularities such as mouse bites.

4. Reference-Based Image Subtraction

Defect localization is achieved by computing the absolute difference:

$$D(x,y) = |T(x,y) - I(x,y)|$$

Where:

- $T(x,y)$ = Template image pixel
- $I(x,y)$ = Test image pixel

This operation highlights structural differences caused by defects.

5. Thresholding and Noise Removal

Raw difference images often contain illumination artifacts. Adaptive thresholding was employed to generate a binary mask. Unlike global thresholding, adaptive thresholding computes localized thresholds, improving detection under uneven lighting conditions.

Morphological operations such as opening and median filtering were applied to remove noise and refine defect boundaries.

6. Output

The final output of this module is a clean binary mask highlighting defect regions. This mask forms the foundation for contour-based ROI extraction in Module 2.

Module 2: Contour Detection and Region of Interest (ROI) Extraction

1. Objective

The primary objective of this module is to accurately isolate individual defect regions from the binary defect mask generated in Module 1 and prepare standardized Regions of Interest (ROIs) suitable for deep learning classification. Precise localization at this stage is critical because classification performance heavily depends on the quality and relevance of extracted defect regions.

2. Contour Detection

After generating the binary mask through image subtraction and thresholding, contour detection is performed using OpenCV's external contour retrieval method. The RETR_EXTERNAL mode is selected to identify only the outermost contours, ensuring that each detected contour corresponds to a distinct defect region. This prevents nested contour detection and simplifies bounding box generation.

Each contour represents a connected component in the binary mask, which may correspond to a valid defect or noise artifact.

3. Contour Filtering and Validation

To eliminate noise and prevent false positives, contour filtering based on area thresholding is applied. Contours with area below a predefined minimum threshold are discarded, as they typically represent insignificant pixel variations rather than actual defects. Additional filtering based on bounding box width and height ensures that extremely small or irregular shapes are excluded.

4. Bounding Box Generation and ROI Extraction

For each validated contour, a minimum enclosing rectangular bounding box is computed. To preserve contextual trace information, padding is added around the bounding box before cropping. This ensures that surrounding copper traces are included, which assists the classification model in learning structural patterns.

The extracted ROI is resized to 128×128 pixels to maintain consistency with the EfficientNet input requirements. All ROIs are stored in class-specific directories, forming a structured dataset for training.

This module ensures precise defect localization, structured data preparation, and optimized input generation for the classification stage.

Milestone 2: Model Training and Evaluation

Module 3: Transfer Learning-Based Defect Classification Using EfficientNet

1. Objective

The objective of Module 3 is to design, implement, and train a deep learning-based classification model capable of accurately categorizing PCB defects into six predefined classes: *Missing Hole, Mouse Bite, Open Circuit, Short, Spur,* and *Spurious Copper*. This module focuses on leveraging transfer learning to achieve high classification performance while maintaining computational efficiency.

2. Model Architecture and Selection

EfficientNet-B0 was selected as the backbone architecture due to its compound scaling strategy, which balances network depth, width, and resolution to achieve superior accuracy with fewer parameters. Compared to traditional CNN architectures, EfficientNet provides better performance-to-complexity ratio, making it suitable for real-time inspection systems.

Pretrained ImageNet weights were utilized to initialize the network, enabling the model to benefit from previously learned feature representations such as edges, textures, and structural patterns. The final fully connected classification layer was modified to output six classes corresponding to the PCB defect categories.

3. Data Preprocessing and Augmentation

All extracted ROIs were resized to 128×128 pixels to match the network's input dimensions. Pixel normalization was performed using standard ImageNet mean and standard deviation values.

To improve generalization and reduce overfitting, data augmentation techniques were applied during training, including:

- Random horizontal and vertical flips
- Small-angle rotations
- Brightness and contrast adjustments

These augmentations simulate real-world inspection variations such as orientation changes and illumination differences.

4. Training Configuration

The model was trained using the Adam optimizer with a low learning rate to ensure stable convergence. Cross-entropy loss was employed as the objective function for multi-class classification. Early stopping and learning rate scheduling were incorporated to prevent overfitting and optimize training efficiency.

5. Performance Outcome

The trained model achieved high classification accuracy on the validation dataset, exceeding the required 95% benchmark. Training and validation curves demonstrated stable convergence, indicating strong generalization capability and reproducible performance.

This module establishes a robust and scalable defect classification framework suitable for deployment within the complete PCB inspection pipeline.

Module 4: Model Evaluation and Prediction Testing

1. Objective

The primary objective of Module 4 is to rigorously evaluate the trained EfficientNet-based classification model and validate its performance on unseen data. This stage ensures that the model not only performs well during training but also generalizes effectively to new PCB defect images, meeting industrial reliability standards.

2. Evaluation Metrics

To comprehensively assess model performance, multiple quantitative metrics were employed:

- **Accuracy** – Overall proportion of correctly classified samples
- **Precision** – Correct positive predictions per class
- **Recall** – Ability to correctly detect actual defects
- **F1-Score** – Harmonic mean of precision and recall
- **Confusion Matrix** – Class-wise performance visualization

These metrics provide deeper insight beyond overall accuracy, particularly for evaluating class imbalance and misclassification behavior.

3. Experimental Results

The trained model achieved approximately **99% classification accuracy** on the validation/test dataset, exceeding the required ≥95% benchmark. Class-wise precision and recall values were consistently high across all six defect categories, demonstrating balanced performance.

The confusion matrix analysis indicated minimal inter-class confusion. For example, structurally similar defects such as *Spur* and *Open Circuit* were accurately distinguished, confirming the model's ability to capture fine-grained visual differences.

4. Testing on Unseen Images

To further validate robustness, the model was tested on unseen ROI samples generated from new PCB image pairs. Predictions were compared against ground truth labels extracted during ROI segmentation.

The prediction match rate remained consistently high, with very low false positive and false negative rates. This confirms that the model generalizes well beyond the training dataset.

5. Stability and Reproducibility

Repeated training runs demonstrated stable convergence patterns, with similar validation accuracy levels achieved across experiments. Early stopping and learning rate scheduling contributed to consistent performance and prevented overfitting.

Overall, this module confirms that the developed classification system meets the performance criteria required for reliable automated PCB defect inspection and is suitable for integration into a real-time inference pipeline.

Milestone 3: Frontend and Backend Integration

Module 5: Web-Based User Interface for Image Upload and Visualization

1. Objective

The objective of Module 5 is to design and implement a user-friendly web interface that enables real-time PCB defect detection and visualization. The interface serves as the interaction layer between the user and the backend defect detection pipeline, allowing non-technical users to upload PCB images and receive annotated defect predictions without manual intervention.

2. Framework Selection

Streamlit was selected as the frontend framework due to its simplicity, rapid development capability, and seamless integration with Python-based backend logic. Streamlit enables the creation of interactive web applications without requiring extensive frontend development using HTML, CSS, or JavaScript.

3. Interface Design and Features

The web interface includes the following functional components:

- Upload field for **template (golden) PCB image**
- Upload field for **test PCB image**
- Automatic trigger for backend inference pipeline
- Display of annotated output image with bounding boxes
- Display of predicted defect labels with confidence scores
- Option to download annotated image and prediction logs

The layout follows a structured workflow:

Upload → Process → Visualize → Export

This ensures intuitive user interaction and minimal operational complexity.

4. Real-Time Visualization

After image upload, the backend processing pipeline is executed automatically. The annotated output image is displayed within the interface, showing:

- Bounding boxes around detected defects
- Class labels
- Confidence percentages

This visual feedback enhances interpretability and helps validate prediction correctness.

5. Usability and Responsiveness

The application was tested on multiple image pairs to ensure:

- Smooth upload functionality
- No rendering issues
- Low latency during inference
- Stable backend connectivity

The average processing time per image pair remains within acceptable limits, supporting near real-time inspection.

6. Significance

This module transforms the defect detection system from a research prototype into a deployable inspection tool. By providing an accessible web interface, the system becomes suitable for practical industrial usage, enabling efficient and automated PCB quality control.

B. Module 6: Backend Pipeline for Image Processing and Model Inference

1. Objective

The objective of Module 6 is to develop a modular and efficient backend inference pipeline that integrates image preprocessing, defect localization, and deep learning-based classification into a unified system. This backend serves as the computational core of the PCB defect detection framework, enabling automated processing of uploaded images from the web interface.

2. Modular Architecture Design

To ensure scalability and maintainability, the backend was structured into independent functional modules:

1. **Template Matching Module** – Identifies the most appropriate defect-free template corresponding to the uploaded test image.
2. **Image Subtraction Module** – Performs reference-based subtraction and generates a binary defect mask.
3. **Contour Detection Module** – Extracts bounding boxes around localized defect regions.
4. **Classification Module** – Uses the trained EfficientNet model to classify each extracted ROI.

This modular design allows easy debugging, future upgrades, and potential integration of advanced segmentation models.

3. Inference Workflow

Upon receiving template and test images from the frontend, the backend executes the following sequential operations:

- Convert images to grayscale and align dimensions
- Perform contrast enhancement using CLAHE
- Compute absolute difference and generate binary mask
- Apply contour detection to identify defect regions
- Extract padded ROIs and resize to 128×128
- Normalize inputs and pass through EfficientNet classifier
- Generate predicted labels with confidence scores
- Overlay bounding boxes and annotations on the original test image

The final annotated image and structured prediction logs are returned to the frontend for visualization.

4. Model Integration

The trained model checkpoint is loaded during initialization to avoid repeated loading overhead. The inference mode disables gradient computation to optimize speed and memory efficiency.

5. Performance Optimization

The pipeline was optimized to reduce latency by:

- Using efficient OpenCV operations
- Avoiding redundant preprocessing steps
- Running inference in evaluation mode

Average processing time per image remains within practical industrial limits, supporting near real-time inspection.

6. Significance

This backend module ensures seamless integration between traditional image processing techniques and deep learning classification. It transforms the research model into a deployable system capable of automated PCB inspection with minimal human intervention.

Milestone 4: System Finalization and Deployment

Module 7: Testing, Evaluation, and Exporting Results

1. Objective

The objective of Module 7 is to finalize the PCB defect detection system by conducting comprehensive testing, performance validation, and implementing result export functionality. This stage ensures that the developed application is reliable, efficient, and suitable for practical deployment in industrial quality inspection workflows.

2. System Testing on Multiple Image Pairs

The complete pipeline was tested on multiple template–test image pairs across all six defect categories. Testing was conducted to verify:

- Correct template selection
- Accurate defect localization
- Proper bounding box generation
- Correct classification labels with confidence scores
- Stable frontend–backend communication

The system demonstrated consistent performance across varying defect sizes and image orientations.

3. Performance Evaluation and Optimization

To meet deployment requirements, inference speed and responsiveness were analyzed. The backend was optimized by:

- Running the model in evaluation mode (disabling gradients)
- Loading the trained model once during initialization
- Minimizing redundant image conversions
- Applying contour filtering to remove noise

The average upload-to-output time remained within acceptable limits for near real-time inspection applications.

4. Export Functionality Implementation

To enhance usability and documentation support, export features were integrated into the web interface. The system provides:

- Download option for annotated output image
- Downloadable CSV file containing prediction logs
- Confidence scores and bounding box coordinates

This enables traceability, audit documentation, and further analytical review.

5. Validation of Industrial Readiness

The finalized system demonstrates:

- High classification accuracy
- Low false positive and false negative rates
- Stable inference performance
- User-friendly output export capability

The integration of export functionality transforms the system from a research prototype into a practical inspection tool capable of supporting industrial PCB quality assurance processes.

Module 8: Documentation and Final Presentation

1. Objective

The objective of Module 8 is to formally document the design, implementation, evaluation, and deployment of the PCB Defect Detection and Classification System. This module ensures

that the developed system is well-structured, reproducible, and presentation-ready for academic evaluation and professional demonstration.

2. Technical Documentation Preparation

Comprehensive technical documentation was prepared to describe:

- System architecture
- Dataset preparation methodology
- Image preprocessing and subtraction techniques
- ROI extraction process
- Deep learning model training and evaluation
- Backend inference pipeline
- Frontend integration and deployment

Each milestone and module was documented with clear objectives, methodology, implementation details, and performance outcomes. Evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis were included to support quantitative validation.

The documentation follows an academic structure suitable for submission and archival purposes.

3. GitHub Repository Organization

A structured GitHub repository was created to maintain professional code organization. The project was divided milestone-wise, with separate folders for:

- Milestone 1 (Image Processing)
- Milestone 2 (Model Training & Evaluation)
- Milestone 3 (Frontend & Backend Integration)
- Milestone 4 (Finalization & Documentation)

The repository includes source code, trained model checkpoint, evaluation outputs, README files, requirements.txt, and proper dependency documentation. This ensures reproducibility and ease of setup.

4. User Guide and System Usage

A user guide was developed describing:

- Installation steps
- Dependency setup
- Running the Streamlit application
- Uploading template and test images

- Interpreting output predictions
- Exporting annotated results

This enables users to operate the system without technical expertise in deep learning or image processing.

5. Final Presentation Preparation

A structured presentation was created covering:

- Problem statement
- Methodology
- Model architecture
- Experimental results
- System demonstration
- Future scope

This ensures the project is demo-ready and suitable for academic defense or industrial showcase.

USER GUIDE

PCB Defect Detection and Classification System

1. Introduction

This user guide provides step-by-step instructions for installing, running, and operating the PCB Defect Detection and Classification System. The system enables automated detection, localization, and classification of PCB defects using image processing and deep learning techniques through a web-based interface.

The application allows users to upload a defect-free template PCB image and a defective test image, and receive annotated results highlighting defect regions along with predicted defect categories.

2. System Requirements

Software Requirements

- Python 3.9 or 3.10
- pip package manager
- Git (optional, for cloning repository)

Required Python Libraries

All dependencies are listed in the requirements.txt file. Main libraries include:

- torch
- torchvision

- opencv-python
- numpy
- streamlit
- pillow
- matplotlib
- seaborn
- scikit-learn

3. Installation Instructions

Step 1: Clone or Download Repository

If using Git:

```
git clone https://github.com/yourusername/PCB_DEFECT_DETECTION.git
cd PCB_DEFECT_DETECTION
```

Or download the ZIP file and extract it.

Step 2: Create Virtual Environment (Recommended)

```
python -m venv pcb-env
pcb-env\Scripts\activate   (Windows)
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

4. Running the Application

Navigate to the Milestone 3 folder (or project root containing app.py) and run:

```
streamlit run app.py
```

After execution, the application will open automatically in your default web browser (usually at http://localhost:8501).

5. How to Use the Application

Step 1: Upload Template Image

Upload the defect-free (golden) PCB image.
This image is used as the reference for subtraction and defect detection.

Step 2: Upload Test Image

Upload the defective PCB image that needs inspection.

The system will automatically:

- Align images
- Perform image subtraction

- Generate defect mask
- Detect contours
- Extract ROIs
- Classify defects using EfficientNet

6. Understanding the Output

After processing, the system displays:

1. Annotated PCB Image

- Bounding boxes around detected defects
- Defect category labels
- Confidence scores (%)

2. Prediction Log

A structured list showing:

- Defect label
- Confidence score
- Bounding box coordinates

7. Exporting Results

The application provides:

- Download button for annotated image
- Download button for CSV prediction log

These files can be used for documentation, reporting, or further analysis.

8. Supported Defect Categories

The system classifies the following defect types:

1. Missing Hole
2. Mouse Bite
3. Open Circuit
4. Short
5. Spur
6. Spurious Copper

9. Best Practices for Accurate Detection

- Ensure template and test images correspond to the same PCB layout.
- Use images of similar resolution.
- Avoid extreme lighting differences.
- Ensure images are not blurred or heavily compressed.

10. Troubleshooting

Issue: No defects detected

- Ensure correct template image is selected.
- Check image alignment.

Issue: Model file not found

- Verify that the trained model file (e.g., pcb_final_model.pth) exists in the project directory.

Issue: Module not found error

- Ensure virtual environment is activated.
- Reinstall dependencies using requirements.txt.

11. System Limitations

- Requires correct template–test pairing.
- Performance depends on image quality.
- Designed for DeepPCB dataset structure.

12. Contact / Repository Information

Refer to the GitHub repository README for:

- Code structure
- Model details
- Experimental results
- Technical documentation

Conclusion

- This project presents a comprehensive and automated PCB Defect Detection and Classification System integrating classical image processing techniques with deep learning-based classification. The developed system successfully combines reference-based image subtraction, contour-based defect localization, and transfer learning using EfficientNet to achieve high-performance defect recognition.
- In Milestone 1, robust image preprocessing and subtraction techniques were implemented to accurately generate defect masks from paired template and test images. Adaptive thresholding and morphological filtering ensured clean defect localization while minimizing noise. Contour detection and ROI extraction further refined defect regions and prepared standardized inputs for deep learning classification.
- In Milestone 2, a transfer learning-based EfficientNet model was trained using extracted defect ROIs. Through systematic preprocessing, augmentation, and optimized training strategies, the model achieved approximately 99% classification accuracy, exceeding the required performance benchmark. Comprehensive evaluation using precision, recall, F1-score, and confusion matrix analysis demonstrated strong generalization capability and minimal inter-class confusion.
- In Milestone 3, the system was transformed into a deployable application through a Streamlit-based frontend and a modular backend inference pipeline. The integrated system allows users to upload PCB images and receive real-time annotated outputs with defect labels and confidence scores. Export functionality ensures traceability and documentation readiness.
- Finally, Milestone 4 ensured system validation, performance optimization, documentation preparation, and repository organization for reproducibility and presentation.
- Overall, the developed system demonstrates high reliability, scalability, and practical applicability for automated PCB quality inspection. By integrating traditional computer vision with modern deep learning techniques, the project successfully delivers a professional-grade, end-to-end inspection framework suitable for academic and industrial deployment.